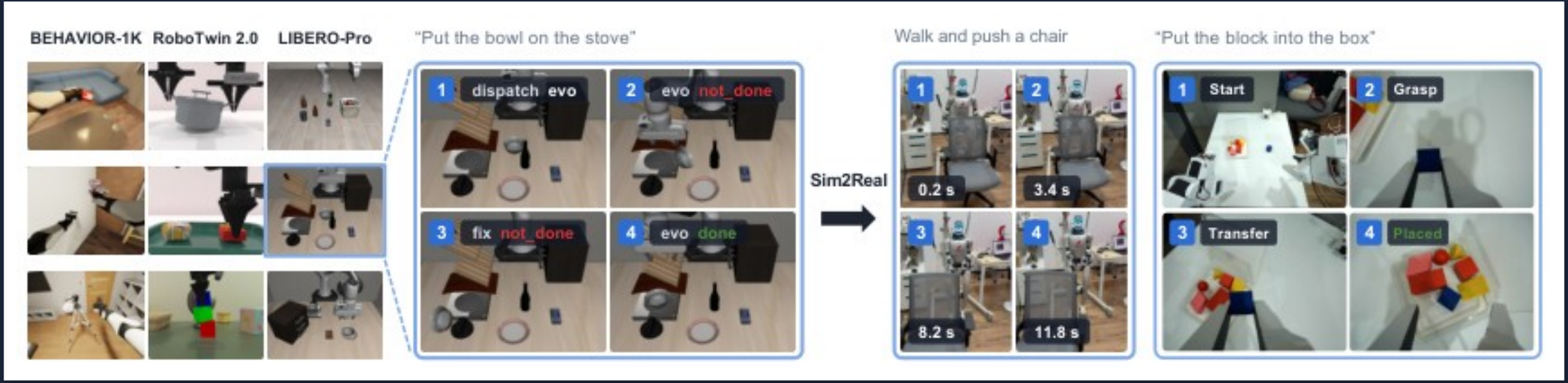
REA： 让慢大脑不再拖住快小脑

REA: A Routed Embodied Agent Runtime for Asynchronous Brain-Cerebellum Control

Pengfei Zhou, Yixing Ma, Zhengfeiyang Zhang, Haoyuan Chen, Zhenglin Wan, Hexin Wang, Wangbo Zhao, Yang You
NUS HPC-AI Lab · InfRec · UC Berkeley · U Tokyo · SIAT CAS · HKUST



论文解读 | arXiv 2610.42053 （2026-10-30） | 图：论文 Fig. 2

一句话总结

大脑离线备好策略、只在边界判定，路由器查表派发

REA 是一个「大脑 System 2 – 路由器 System 1 – 小脑 System 0」三层具身运行时：大脑离线生成策略、在线只在策略返回边界判定完成；路由器按开发集实测成功率查表派发，并在共享预算内逐级升级恢复——**运行中的策略永远不等大脑。**

95.9%

LIBERO-Pro 成功率
最强单方法 94.7%

−17%

控制时间 vs Agents SDK
−25% vs 阻塞式 advisor

86%

RoboTwin VLA + 代码
最强单 VLA 80%

8/10

真机 G1 推椅子
最佳基线 5/10

贡献定位：系统与评测层面——统一的三层契约 + 路由器三模块（route memory / judge gate / escalation ladder）+3 万余条对齐测试 episode 。

大脑会想但太慢，小脑够快但太脆，两者还没法放在一起

System 2 大脑：推理强，但慢且贵

43 min

ASPIRE 修复循环
每个 LIBERO 任务（中位数）

89M

Codex + GPT-6 为 60 任务
生成程序的输入 token

1 / 1,987

逐 token 输出动作的 VLM
在 LIBERO-Pro 上的成功数

0 / 600

3 cm / 15° 笛卡尔 IK 接口
直接控制的成功数

System 0 小脑：快，但泛化差

VLA / 全身控制器

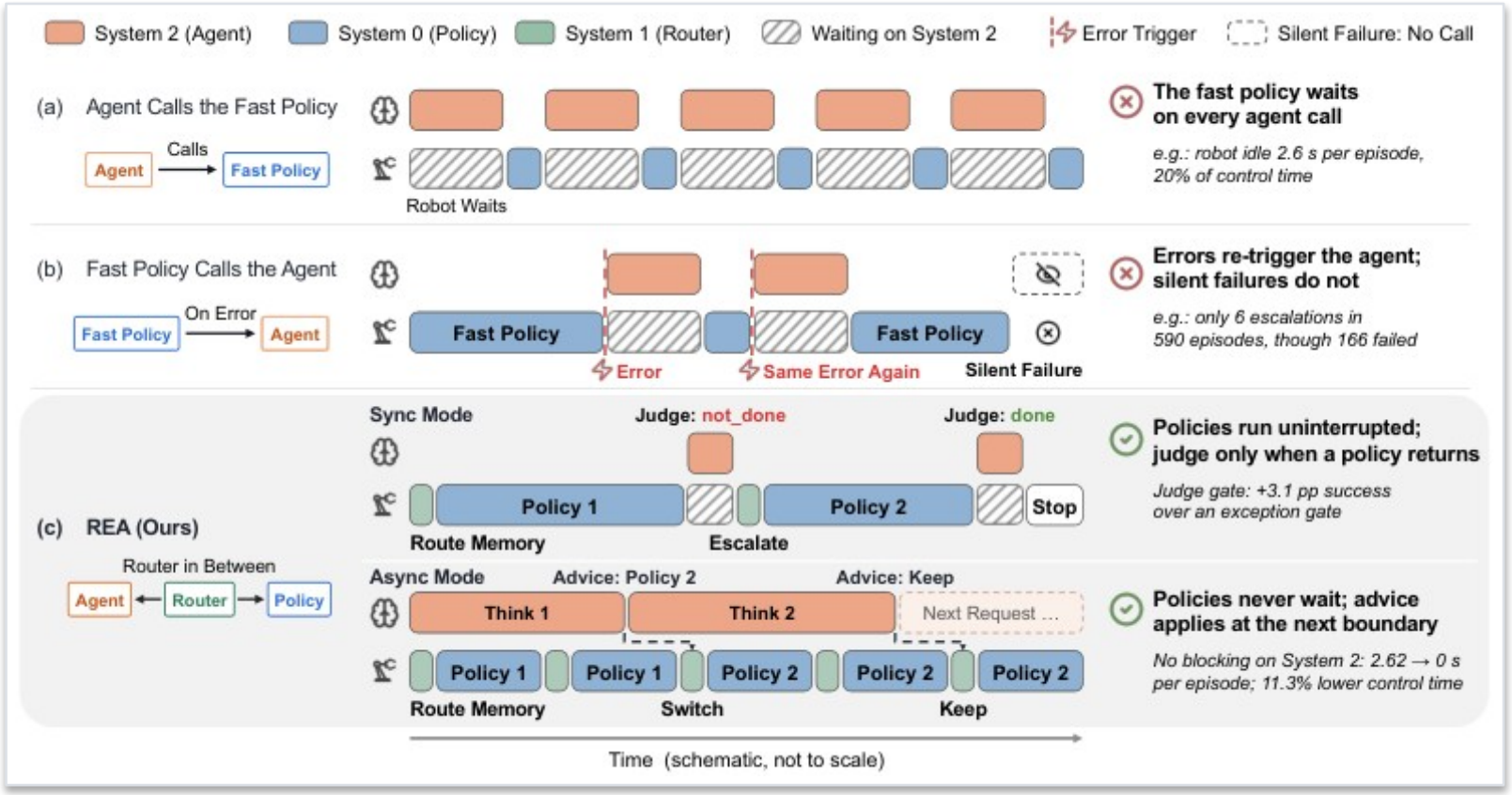
- 运行在几 Hz 到几十 Hz，远快于 agent 的思考
- 在 LIBERO 上超过 90% 的 VLA，在扰动版 LIBERO-Pro 上崩溃
- 适合做「反射」，不适合独自应对分布外任务

障碍 1 没有共同运行时： agent 与 VLA 各用各的模拟器、观测格式、成功判据，无法在同一批 episode 上比较。

障碍 2 异步协调难：大脑调小脑会让执行停摆；小脑出错再叫大脑，会反复触发，又漏掉不报错的失败。

核心问题

两种常见耦合都失败：要么等大脑，要么看不见静默失败



(a) Agent 调快策略

机器人每次都等：空等 2.6 s / episode，占控制时间 20%

(b) 快策略出错叫 Agent

590 ep 中 166 个失败，异常只触发 6 次升级——多数失败是「静默」的

(c) REA

查表派发、策略原生频率运行；只在策略返回时判定 done / not_done

方法：三层契约

三层各司其职：大脑出主意，路由器管控制流，策略只管执行

System 2 · 大脑

编码 agent / VLM

离线 Generate(t) 生成候选策略；在线 Judge(f_0, f_k, ℓ) → done / not_done ，Advise(o) 推荐候选

System 1 · 具身路由器

不输出任何动作

决定跑哪个策略、何时咨询大脑、何时停止；从不阻塞正在运行的策略

System 0 · 小脑

代码策略 / VLA / 全身控制器

以原生频率闭环运行，在 segment 或 policy 边界把控制权交回路由器

统一执行接口

$(\Delta, \delta, e, \tau) \leftarrow \text{Run}(p; H-u, W-w)$

只拿剩余步数 / 墙钟预算；返回本次消耗、异常 e 、截断标志 τ

三个实验变体

REA-M	route memory 选策略 + 异常门；在线零次调用大脑
REA-A	agent 负责初始派发与异常后的恢复选择
REA-V	route memory + 每次策略返回调用视觉 judge

一个协调问题对应一个模块：派发、判定、恢复

① Route memory

派发不经过大脑

- 开发种子（与测试不相交）上每个候选跑约 10 次
- 记录每任务成功率 $\hat{r}$ 与平均步数 $\hat{c}$
- 测试时 argmax $\hat{r}$，平局取步数少者
- 派发 = 一次查表， 0 token

② Judge gate

在策略返回时抓静默失败

- 每次 Run 返回调用一次视觉 Judge
- done → 停止，保护已完成的工作
- not_done → 升级到下一级
- 对照：免费的异常门，只在异常 / 截断时升级

③ Escalation ladder

恢复次数与预算都有上限

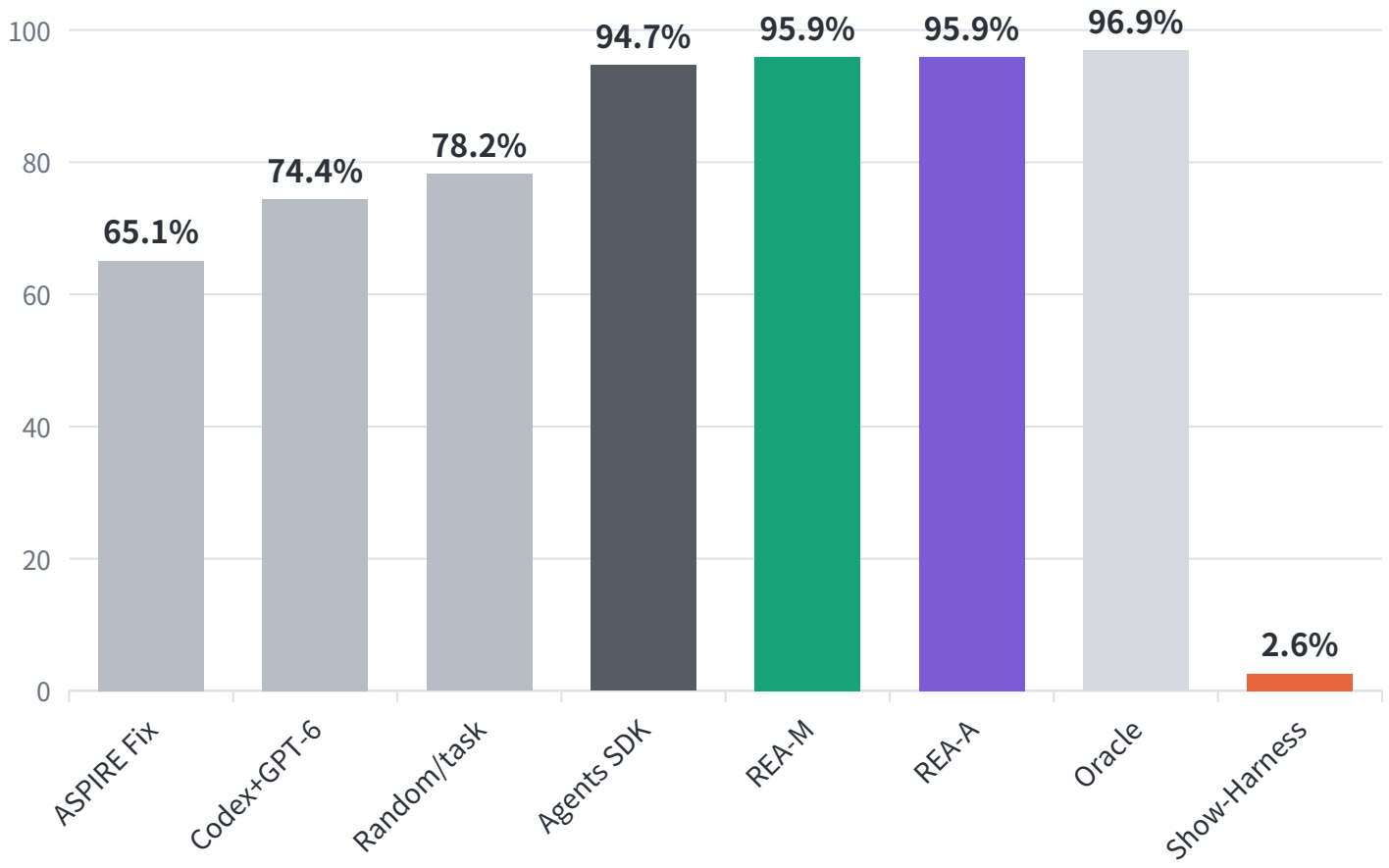
- 阶梯 L = [first, other, first]
- 每级从当前状态继续执行
- 所有运行和大脑调用共享 (H−u, W−w)
- 一个 episode 不超过单策略预算

例（Table 12）： GS-drawer 任务开发集 Fix 1/10 、 Evo 10/10 → 先派 Evo ，阶梯为 [Evo, Fix, Evo] 。

主结果 · LIBERO-Pro

靠路由而不是更大的池子： 95.9% 超过池中每个单一方法

成功率（%）， All 60 tasks



+1.2 pp

vs 最强单方法 Agents SDK （86:51， p=0.004）

95.3 vs 93.7

37 个严格对齐任务（相同程序文件， p=0.001）

1,003 vs 1,343

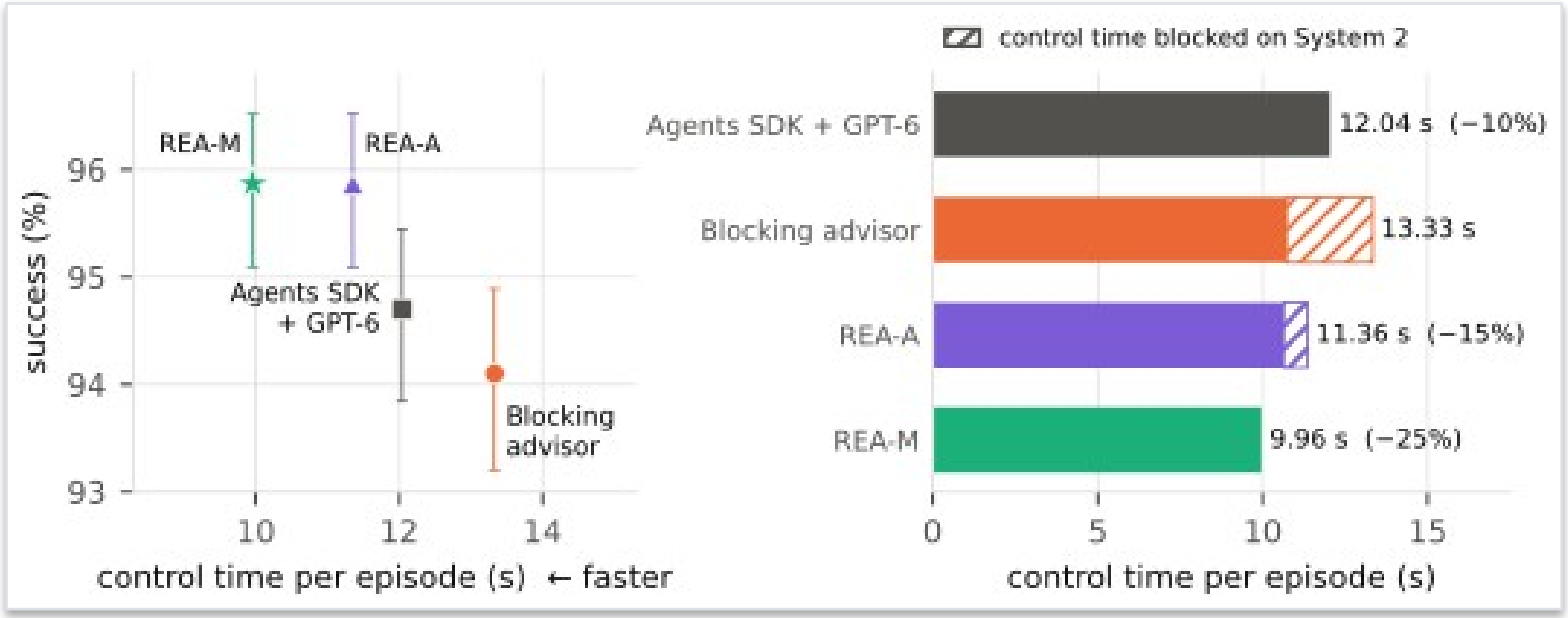
每次成功的控制步数（– 25%）

0 token

route memory 仅用 1,790 开发 episode；
生成流水线各耗 29–89M token

效率

把大脑请出控制环： REA-M 是最快的计时方法，控制时间少 25%



2.62 s

阻塞式 advisor 每 episode 空等时间（占控制时间 20%）

9.96 s

REA-M 每 episode 控制时间（− 25% vs advisor，− 17% vs SDK）

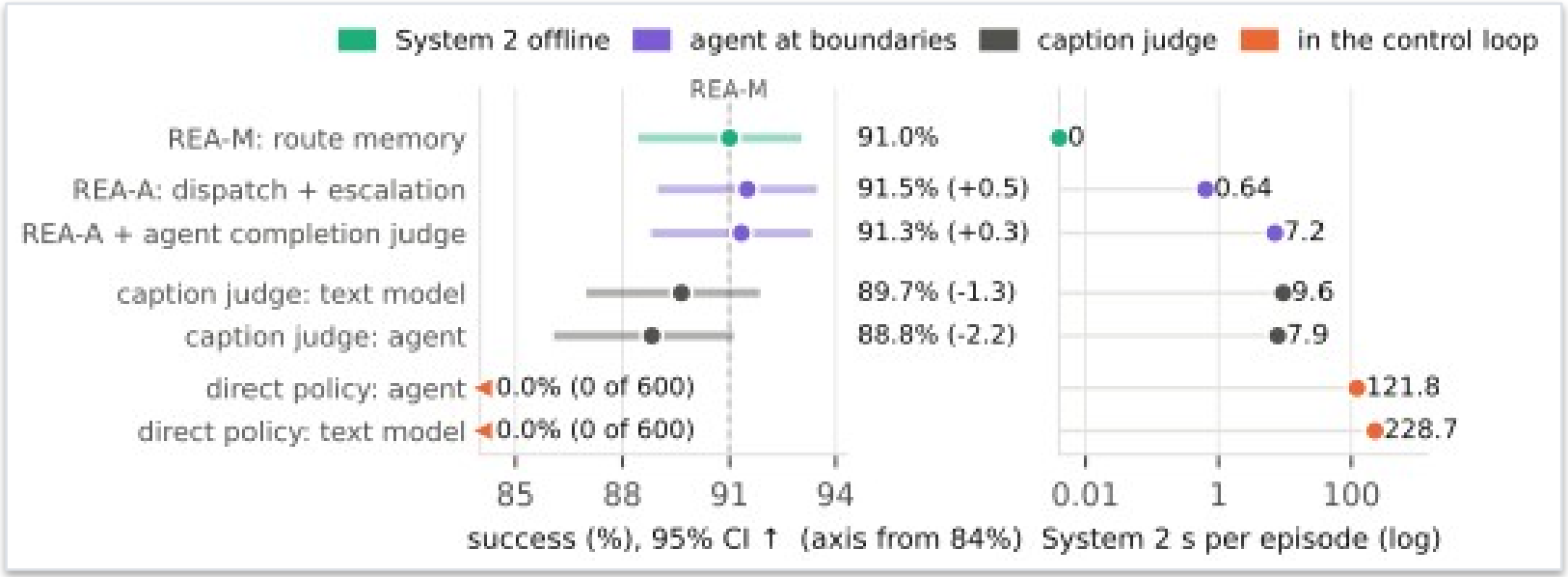
10.3 ms

两者单个控制步耗时相同 → 差距来自等待与步数，而非硬件

阻塞式 advisor 推翻 route memory 选择的 1,236 个 episode 中： route memory 的策略可解 1,168 个， advisor 只解 1,124 个。

System 2 放在哪

大脑放在离线最划算；放进控制环直接归零

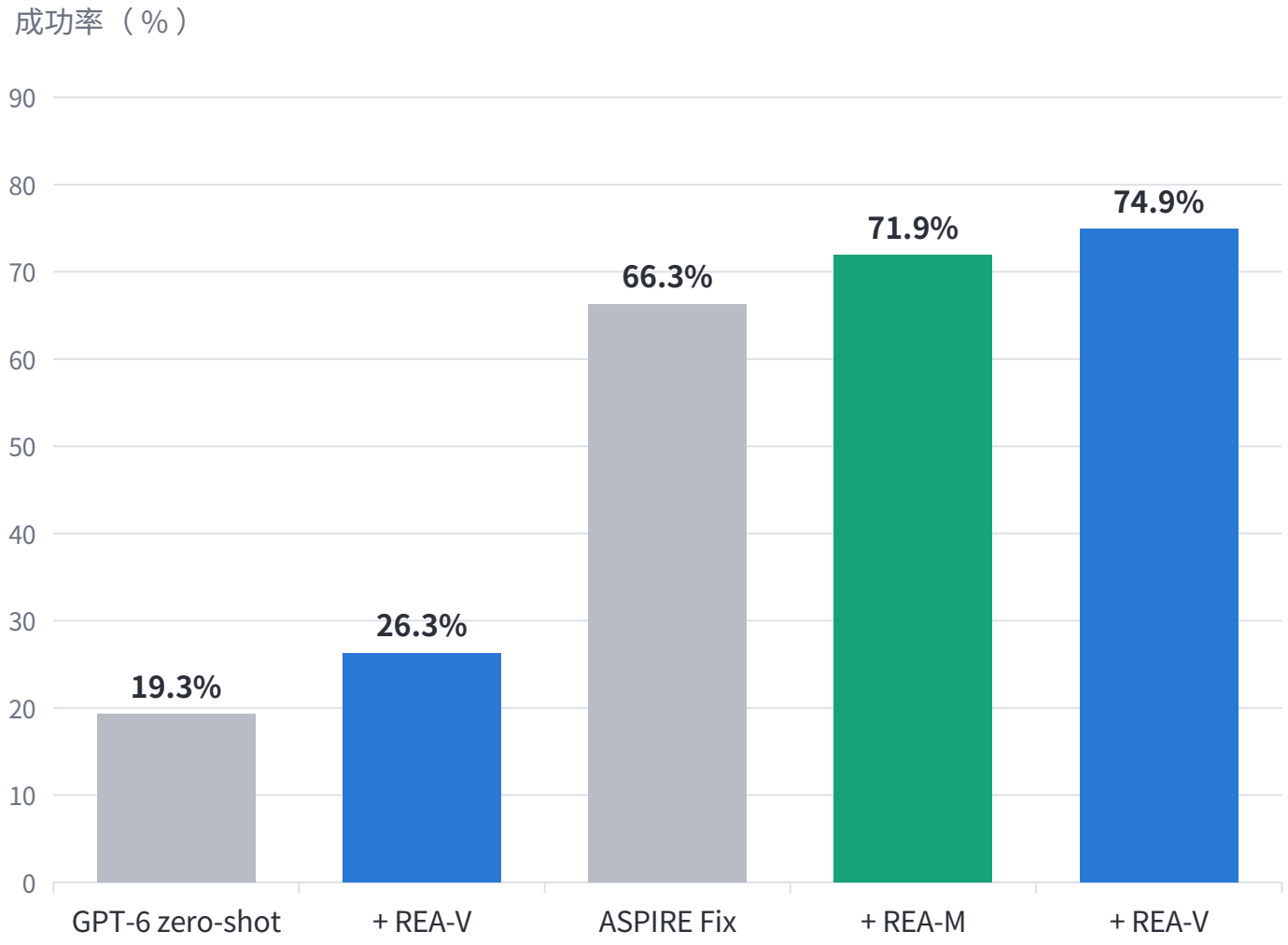


位置	成功率 / 大脑耗时
离线（REA-M）	91.0% 0 s
派发 + 恢复（REA-A）	91.5% 0.64 s
+ agent 完成判定	91.3% 7.2 s
文字描述判定	88.8–89.7% 7.9–9.6 s
直接当控制策略	0.0% 122–229 s

结论：大脑在离线生成策略时价值最大；在线只在「策略会静默失败」时，以视觉 judge 的形式才值得付费。

跨策略库

无论策略库强弱， REA 都能把它往上推



ASPIRE 库： 66.3% → 74.9%

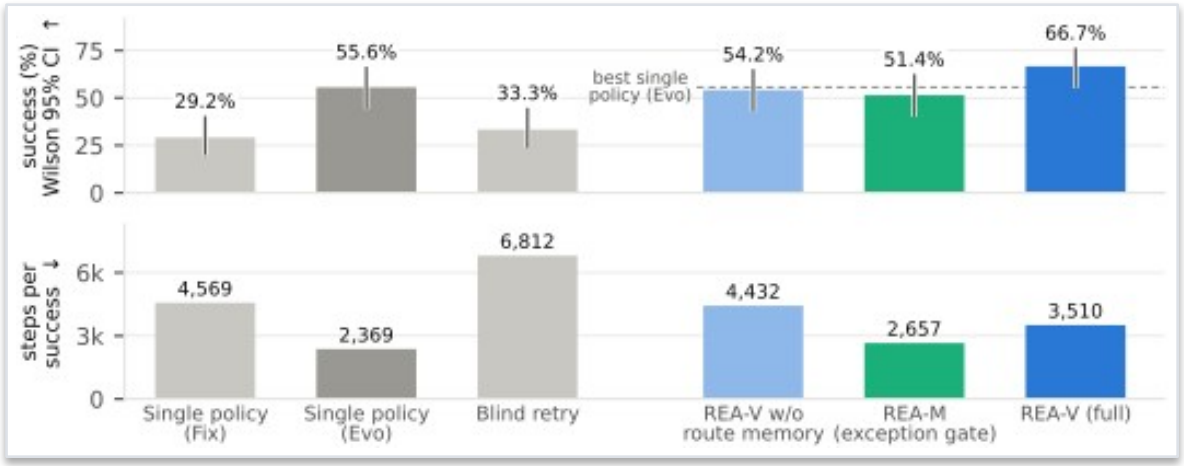
- REA-M +5.6 pp （49:16），步数 / 成功反降：1,557 → 1,467
- 异常门仅触发 6 次 → 增益几乎全部来自 route memory 先派 Evo
- judge gate 再 +3.1 pp （p=0.022）

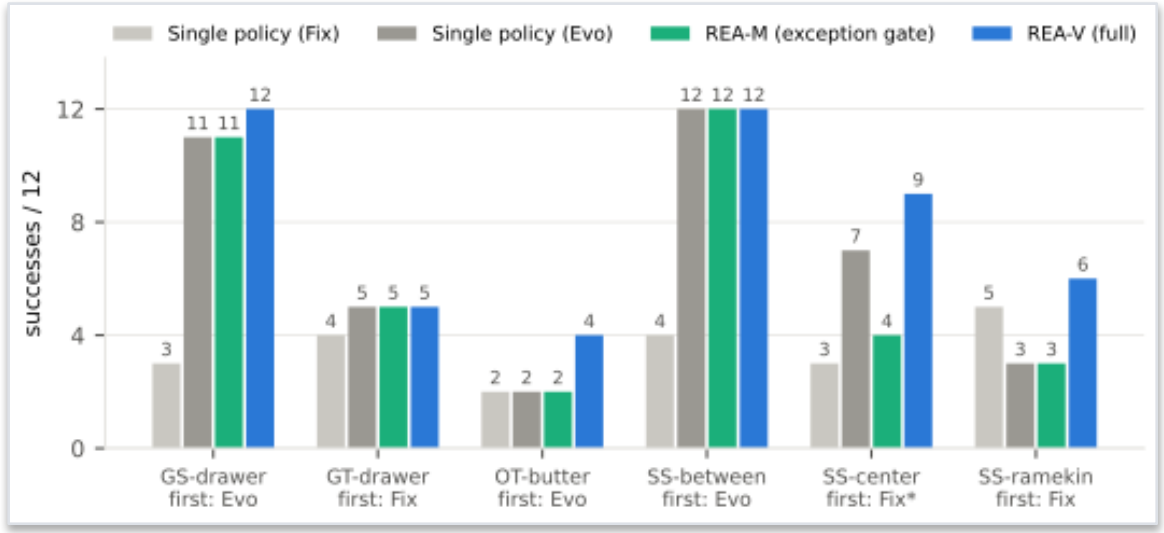
弱库： 19.3% → 26.3%

GPT-6 零样本两个样本间路由 + judge （50:8，$p<10^{-4}$）

代价： judge gate 带来约 2.1× 的每次成功墙钟时间（40.9 → 120.7 s）。

两个模块缺一不可：只有完整路由器超过最佳单策略





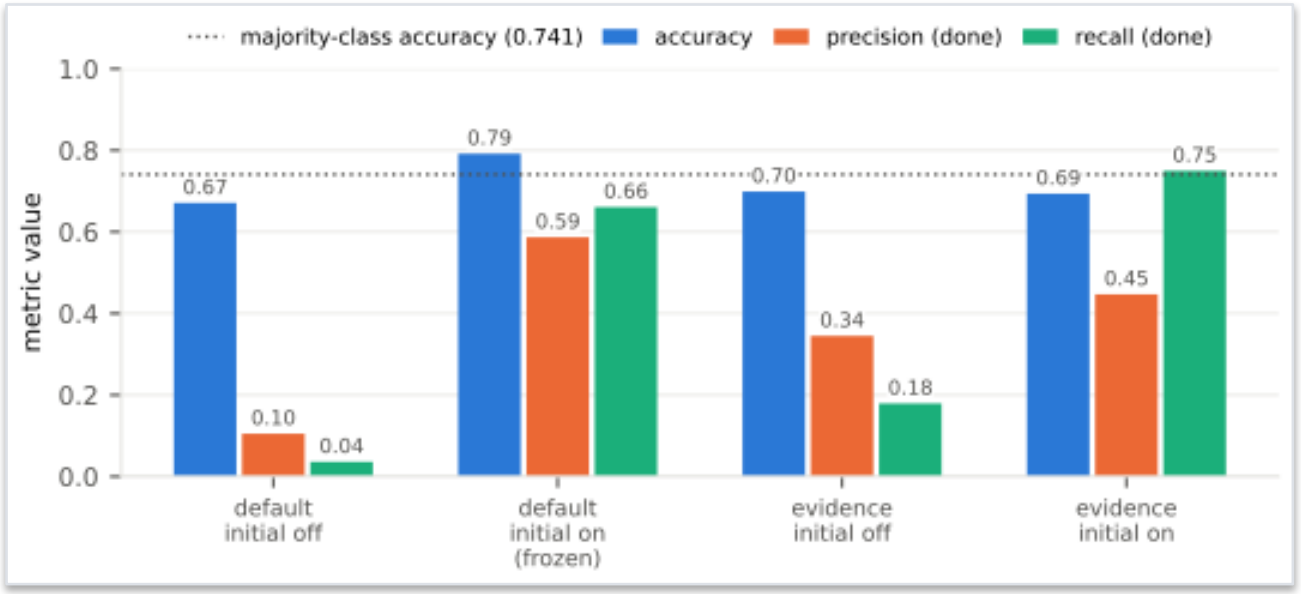
66.7%
完整 REA-V （48/72），最佳单策略 Evo 55.6%

−15.3 pp
把 judge gate 换成异常门（12:1， p=0.003）

33.3%
盲目重跑：无协调恢复几乎无效

机制

噪声 judge 为何仍有用：说 done 时很准，重跑很少破坏成果



校准（216 条开发标签）： 给出初始帧后，准确率 0.67 → 0.79 ，是唯一超过多数类基线 0.741 的配置；阈值 0.5–0.9 结果平稳。

0.83 / 0.905

done 判定精度（困难子集 / LIBERO-Pro ）

24% / 19%

重跑救回的失败首跑比例

5% / 9%

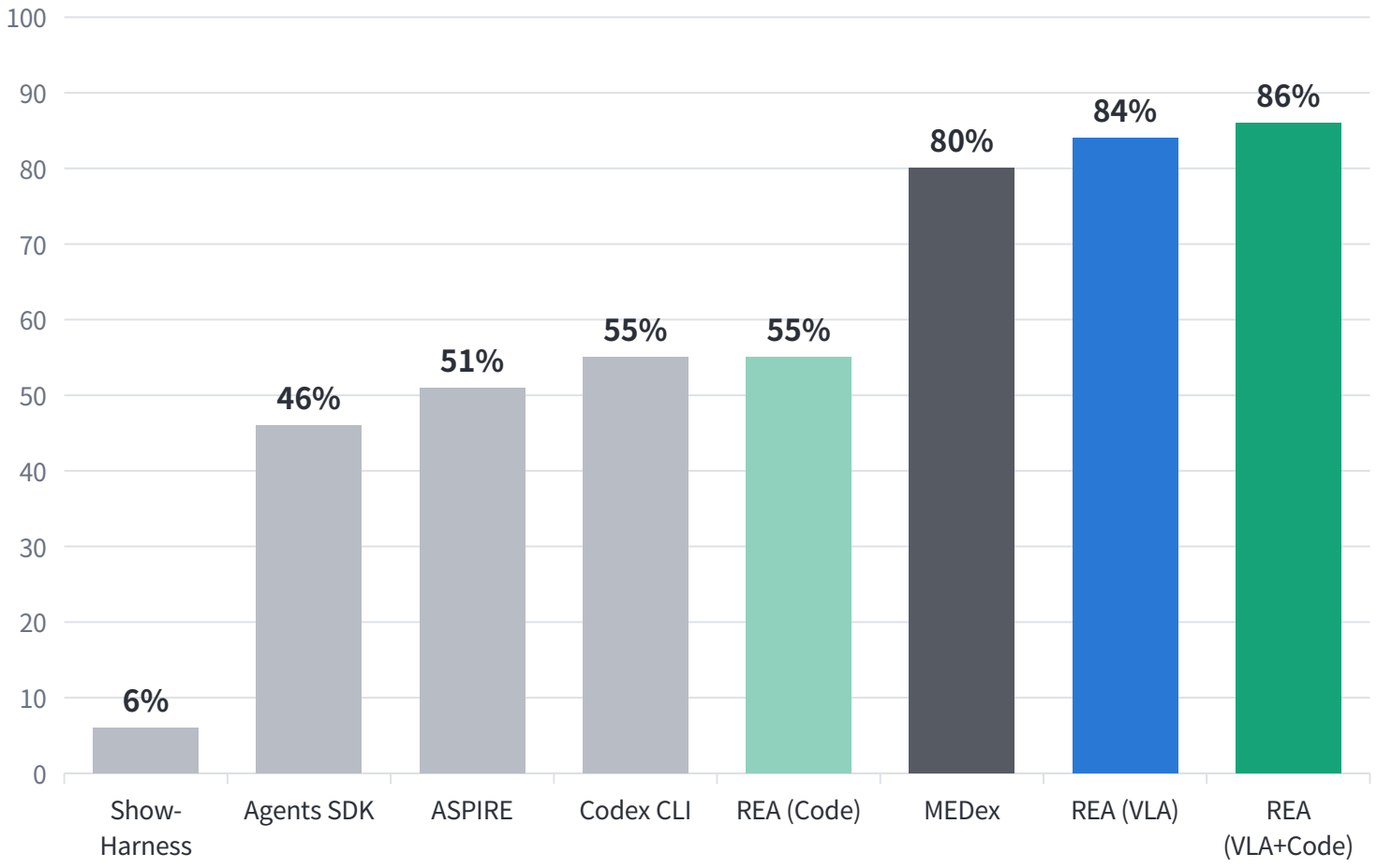
重跑破坏的成功首跑比例

注：最终判定整体准确率仅 0.72 / 0.67 ；作者称这是「一致的解释，而非被检验的机制」。

VLA + 代码

同一接口装下 VLA 与代码策略：混合池 86% 超过最强单 VLA

总体成功率（%）， 100 个场景



学习式 System 1 选择器

- Qwen3.5-4B + LoRA 读初始指令、三视角图像、机器人状态
- 每个候选一个 sigmoid 成功率头，开局取最高并固定整局

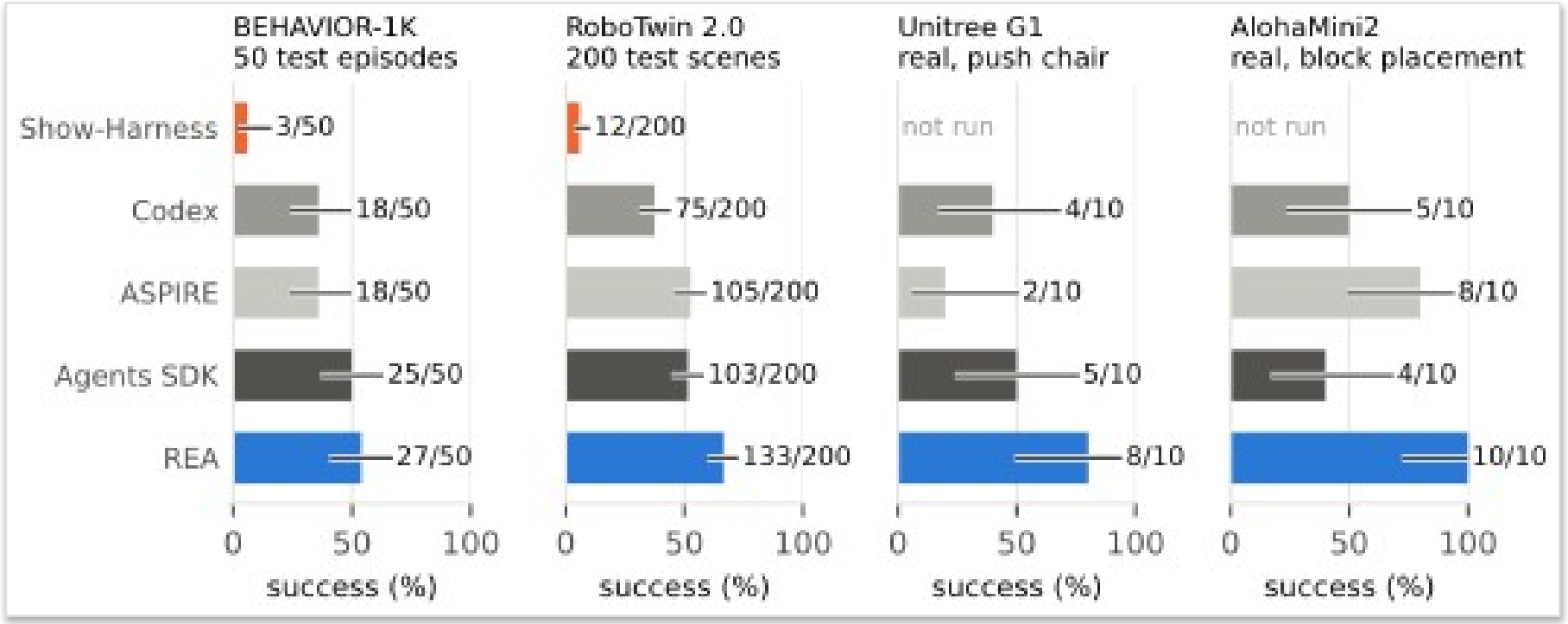
代码策略的贡献

9 个随机化场景选中代码，成功 8 个；净 +2，无丢失

注意：基线为不同时间窗口的历史运行， MEDex 在另一台主机；本实验不含中途切换与 judge gate 。

迁移

换个后端就能用：各迁移场景中 REA 观测成功数均最高



BEHAVIOR-1K

27/50 vs SDK 25/50

差距不显著（7:5， p=0.77），主要来自 camera 一个任务

RoboTwin 2.0 · 200 场景

97 vs Codex 75 （同晚）

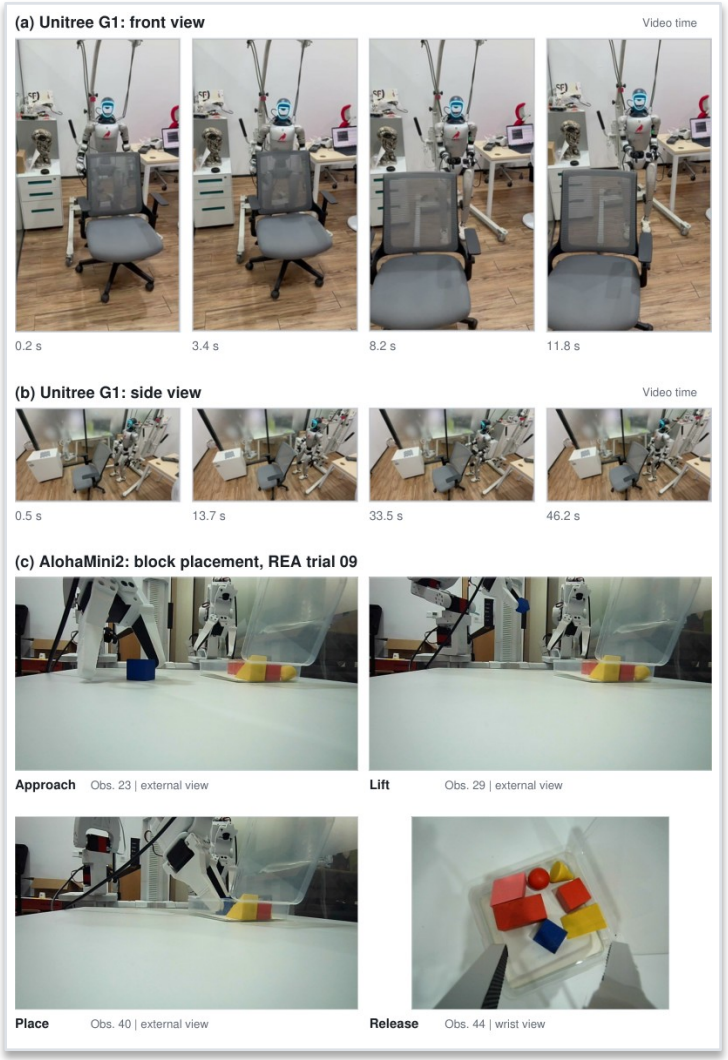
另一晚 REA 133；同配置两晚相差 36 场景 → 存在窗口效应

真机

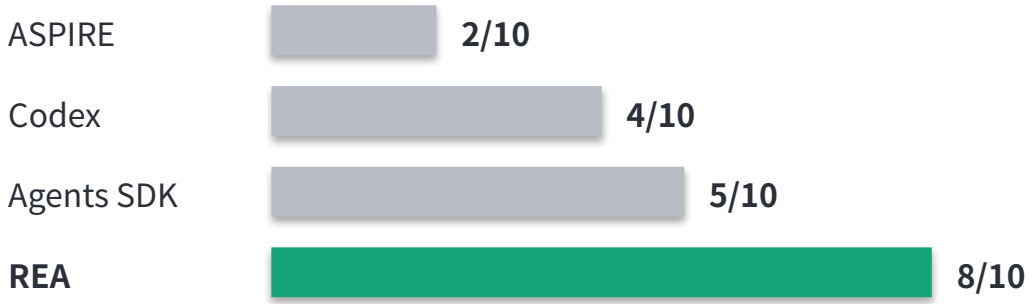
G1 8/10 · Aloha 10/10

每方法 10 次、人工判定； Aloha 上 REA 耗时更长

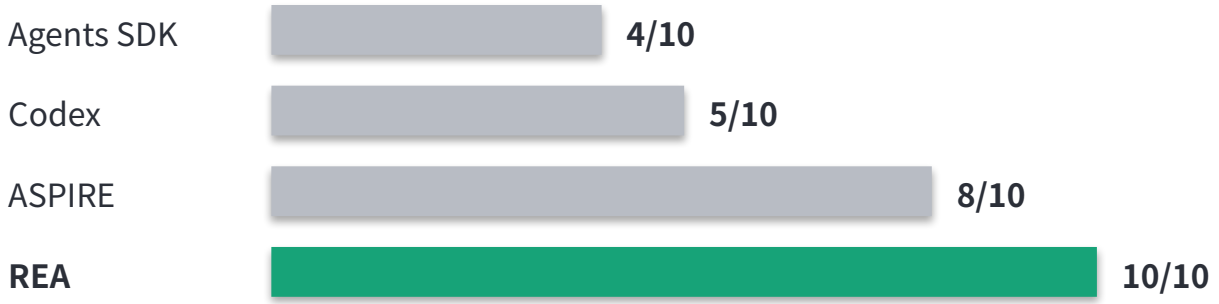
真机上同一运行时： G1 推椅子 8/10 ， π0.5 + 代码路由 8/10



Unitree G1 · 走近并推椅子（操作员现场判定）



AlohaMini2 · 方块放入盒中（四方法，人工复核）



π0.5 单独 3/10 → REA （π0.5 + 代码） 8/10 ，放置失败 5 → 0 ； 但两组不同日期采集，REA 耗时更长。

批判性看待

证据强度不均：主结果扎实但增益小，扩展实验偏弱

01 增益幅度

全集仅比最佳单方法 +1.2 pp； oracle 上限 96.9%。成功率天花板仍由大脑生成的策略库决定（各方法相差 20–47%）。

02 judge gate 的证据范围

全集主结果用的是异常门； judge 的 +15.3 pp 来自专门挑选的困难子集，未挑选种子上为 +3.1 pp，且时间约 2.1×。

03 VLA 实验对照

RoboTwin VLA 基线为不同时间窗口的历史运行， MEDex 在另一主机；只做开局一次性选择，代码净增 2 个成功。

04 样本与波动

BEHAVIOR 50 ep 且对 SDK 不显著；真机每方法 10 次、人工判定； RoboTwin 同配置两晚相差 36 场景。

05 冷启动

route memory 按任务查表，依赖每任务的开发 episode；对全新任务如何派发未讨论（RoboTwin 学习式选择器除外）。

06 异步覆盖

LIBERO 主实验中 judge 在边界同步调用、延迟计入预算；主表未单独报告异步 advisor 的成功率，异步模式数字见 Fig. 3 注释与 AlohaMini2 实验。

带走三句话

1

大脑用在「之前」和「边界」，不要放进运行中的控制环

离线生成策略 + 策略返回时判定；在环咨询只会增加等待（阻塞式 advisor 空等 2.62 s/ep ）。

2

基于实测结果的路由，便宜且有效

route memory 0 token ，就能达到在线 agent 派发同样的 95.9% ，并且最快。

3

静默失败需要边界判定， judge 的关键是 done 判定要准

done 精度 0.83 / 0.905 ；按库选择门：策略少失败用免费异常门，静默失败多时用 judge 。